Worldview OS

Execution document: Product Requirements Document (PRD), system architecture, prompt flow, and MVP build plan

Document purpose	Translate the Worldview Agent concept into a buildable product with clear scope, workflow, and delivery plan.
Recommended product name	Worldview OS — a structured intelligence engine that decomposes any question into lenses, scenarios, indicators, and synthesis.

Executive summary. Worldview OS should not be built as an answer bot. It should be built as a structured intelligence workflow that normalizes questions, routes them through a reusable angle library, generates scenario trees, scores probabilities and impact, and returns a watchlist of signals that would confirm or disprove the thesis.

Contents

1. Product vision and positioning
2. Problem statement
3. Product goals, non-goals, and success metrics
4. Target users and priority use cases
5. Core product requirements
6. System architecture
7. Prompt flow and multi-agent orchestration
8. Data model and storage
9. MVP build plan
10. Risks, controls, and next-phase roadmap

1. Product vision and positioning

Worldview OS is a question-to-judgment platform. The product accepts a natural-language question and converts it into a disciplined analytical output: what the real question is, which lenses matter, what scenarios are plausible, what indicators should be monitored, and how the current balance of probabilities should be scored.

The positioning should be explicit: this is not “AI that knows everything.” It is “a structured intelligence engine that shows every serious angle behind any question.” That distinction matters because the product’s edge is methodology, not generic text generation.

Core value proposition

- Force coverage across multiple analytical lenses instead of allowing a single narrative to dominate.
- Standardize thinking quality across politics, markets, crypto, sports, gaming, and any additional domain plug-in.
- Convert prose into decision-grade outputs: scenarios, indicators, risks, confidence, and change-of-mind conditions.
- Create reusable memory: previous forecasts, prior probabilities, revisions, and observed outcomes.

Product thesis

People do not mainly suffer from lack of information; they suffer from unstructured thinking, narrative capture, and failure to compare alternative pathways. Worldview OS creates leverage by institutionalizing analytical discipline.

2. Problem statement

When users ask broad questions such as “Will a president fail?”, “Is Bitcoin going higher?”, or “Can Team X win the title?”, most AI tools respond with a polished answer that mixes facts, guesses, and vibes. The user may receive a readable paragraph, but not a robust framework for thinking.

- Questions are often ill-posed or ambiguous.
- Relevant domains are missed because generic assistants do not force lens expansion.
- Outputs rarely distinguish between evidence, assumptions, and speculation.
- There is usually no scenario matrix, no watchlist, and no tracked forecast history.
- Users cannot audit why the answer changed over time.

Jobs to be done

User need	Current failure mode	Desired product outcome
Understand a complex issue	Receives a one-dimensional	Receives a map of the issue,

quickly	answer	ranked drivers, and scenarios
Decide what to monitor next	Gets no operational watchlist	Receives leading indicators and disconfirming signals
Compare viewpoints without ideology	Gets narrative-heavy summaries	Receives structured arguments for and against the thesis
Build a durable worldview	Starts from zero every session	Receives persistent memory and forecast revision tracking

3. Product goals, non-goals, and success metrics

Goals

- Normalize any incoming question into a precise analytical target.
- Route the question through a universal angle library and domain-specific plug-ins.
- Produce a scenario matrix with probability, impact, time horizon, confidence, and watchlist indicators.
- Persist analysis state so users can compare new judgments against prior versions.
- Support multiple user objectives: research, investing, operating decisions, content development, and forecasting.

Non-goals for MVP

- Fully autonomous open-ended research without user review.
- Real-time expert-grade prediction accuracy across all domains from day one.
- Native brokerage, trading, betting, or execution functionality.
- A broad consumer social product; the first version should optimize for depth and utility.

Success metrics

- Coverage quality: percentage of outputs judged to include all major relevant lenses.
- Decision utility: percentage of sessions where users rate the watchlist or scenario table as actionable.
- Revision quality: percentage of forecast updates that clearly explain why probabilities changed.
- Retention: weekly return rate among users who save at least one forecast.
- Latency target for MVP: first structured answer in under 20 seconds for standard queries.

4. Target users and priority use cases

Primary users

- Analytical operators: founders, strategists, researchers, policy watchers, and investors who need structured judgment.
- Market participants: equity, macro, and crypto users who care about scenarios and catalysts.
- Generalist knowledge workers: users who think across politics, economics, technology, culture, sports, and gaming.

Priority use cases

Use case	Example query	Expected output	Priority
Politics / geopolitics	What are the main pathways for the current U.S. administration to lose policy momentum by 2027?	Institutional, legal, economic, social, and narrative breakdown with scenario scoring	P0
Markets / crypto	What are the serious bull and bear cases for Bitcoin over the next 12 months?	Macro, liquidity, regulation, positioning, and on-chain scenario tree	P0
Sports	Can a given NBA team win the title?	Roster, injuries, coaching, cap flexibility, schedule, and matchup geometry	P1
Gaming	Will a racing title retain users after launch?	Gameplay loop, monetization, patch cadence, esports/community, and retention risks	P1
Open-ended research	What are all the angles to evaluate a country relocation decision?	Economic, legal, social, career, safety, tax, and lifestyle analysis framework	P1

5. Core product requirements

Functional requirements

1. The system shall rewrite the user query into a precise analytical question and explicitly define ambiguous terms.

2. The system shall classify the query by domain, time horizon, and user objective.
3. The system shall select a universal angle pack plus any relevant domain-specific angle pack.
4. The system shall generate structured arguments for the thesis, against the thesis, and for underappreciated alternatives.
5. The system shall build a scenario matrix with probability, impact, time horizon, confidence, and leading indicators.
6. The system shall return a concise synthesis and a “what would change my mind” section.
7. The system shall store the query, output, scenario scores, and subsequent revisions.
8. The system shall allow follow-up questions that refine or challenge the prior analysis.

Non-functional requirements

1. Traceability: the system must separate observed facts, assumptions, inferences, and speculation.
2. Consistency: similar questions should produce structurally similar outputs.
3. Extensibility: new domains should be added through plug-in angle packs rather than a rewrite of the entire logic.
4. Observability: logs should capture prompt versions, model responses, scoring changes, and failure cases.
5. Cost control: the system must support tiered model usage, with premium reasoning reserved for synthesis or red-team steps.

Universal angle library (MVP baseline)

- Structural/system forces
- Incentives and game theory
- Power and decision rights
- Timing and triggering events
- Historical analogs and base rates
- Market pricing and positioning
- Narrative, media, and perception
- Legal and institutional constraints
- Behavioral and social dynamics
- Second-order effects
- Fragility and tail risks
- Data, indicators, and falsification tests

6. System architecture

The recommended architecture is a pipeline, not a single-agent blob. Each stage has a clear responsibility. This reduces hallucination, improves debugging, and makes domain expansion manageable.

Layer	Component	Responsibility	MVP implementation note
Input	Question intake	Receive raw user question, user objective, and optional domain hint	Web form or chat interface
Routing	Normalizer + classifier	Rewrite the question and determine relevant domains and angle packs	LLM call with deterministic schema
Reasoning	Lens expander	Generate all serious analytical lenses and rank relevance	LLM call using universal and domain templates
Reasoning	Scenario engine	Construct bull, base, bear, and wildcard scenarios	Structured generation with fixed fields
Reasoning	Red team	Attack weak assumptions and surface missing variables	Separate model call
Evidence	Retriever	Pull recent factual context, data, and reference signals	Web/data API layer
Synthesis	Chief of staff	Merge outputs into one decision-grade response	Final higher-quality model call
State	Memory store	Persist queries, outputs, forecasts, revisions, and user preferences	Postgres + vector store optional
Monitoring	Telemetry	Track usage, failures,	Basic app logging and

	model cost, and output quality	analytics

Logical flow

1. User query
2. Question normalization
3. Domain classification
4. Angle-pack selection
5. Lens expansion
6. Scenario generation
7. Evidence retrieval / signal pull
8. Red-team challenge
9. Synthesis and scoring
10. Memory write-back

Suggested MVP stack

- Frontend: simple web app or chat UI.
- Backend: Python with FastAPI.
- LLM orchestration: structured calls with JSON schemas.
- Storage: PostgreSQL for runs, prompts, scenarios, and user history.
- Retrieval: web/news layer plus optional market and sports APIs.
- Background jobs: lightweight task queue for refreshes and scheduled updates.
- Observability: logs, prompt versioning, and token/cost monitoring.

7. Prompt flow and multi-agent orchestration

The prompt stack should be modular. Each agent should have a narrow mandate, fixed output schema, and explicit constraints. This is what keeps the system from degenerating into one long, expensive monologue.

Recommended agent set for MVP

- Normalizer: rewrites the question and defines terms.
- Router: selects relevant domains, time horizon, and angle packs.
- Lens analyst: expands the query into serious analytical lenses.
- Scenario engine: builds scenario matrix and indicator set.
- Red team: identifies blind spots, hidden assumptions, and disconfirming evidence.
- Synthesizer: produces the final answer in one coherent structure.

Prompt flow

Step	Agent	Input	Output schema
1	Normalizer	Raw question, user objective, time horizon if provided	normalized_question, defined_terms, missing ambiguities
2	Router	Normalized question plus angle library metadata	domains, angle_packs, confidence, required data sources
3	Lens analyst	Normalized question plus selected packs	lens list with rationale, ranked importance, key drivers
4	Scenario engine	Lens list and drivers	scenario matrix, probability, impact, confidence, indicators
5	Red team	All prior outputs	counterarguments, blind spots, missing indicators, failure modes
6	Synthesizer	All prior structured outputs plus evidence	final answer, executive summary, watchlist, what changes the view

Example final response schema

```
{
  "normalized_question": "...",
  "definitions": [{"term": "...", "definition": "..."}],
  "domains": ["politics", "economics", "markets"],
  "top_drivers": [{"name": "...", "importance": 0.0, "why_it_matters": "..."}],
  "scenarios": [
    {
      "name": "Base case",
      "description": "...",
      "probability": 0.45,
      "impact": "medium",
      "time_horizon": "6-12 months",
      "leading_indicators": ["...", "..."],
      "disconfirming_signals": ["...", "..."]
    }
  ],
  "arguments_for": ["..."],
  "arguments_against": ["..."],
  "underappreciated_variables": ["..."],
  "what_changes_my_mind": ["..."],
  "synthesis": "..."
}
```

Prompt design principles

- Make each agent do one job only; never ask one prompt to normalize, reason, red-team, and synthesize at once.
- Force explicit field names and typed outputs.
- Require separation between facts, assumptions, and inferences.
- Require “what would falsify this” in every forecasting step.
- Version every prompt; prompt drift without version control will wreck quality over time.

8. Data model and storage

The MVP does not need a giant knowledge graph, but it does need clean state management. The minimum viable data model should allow the team to answer four questions: what was asked, what the system concluded, why it concluded that, and how the view changed later.

Core tables / objects

- users: identity, preferences, saved domains, and default objectives
- queries: raw question, normalized question, timestamp, domain classification
- runs: model versions, prompt versions, latency, token use, and status
- scenario_sets: scenario names, probabilities, impact labels, confidence, and watchlists
- artifacts: final response, structured JSON, citations, and attached evidence
- forecast_revisions: previous scores, new scores, and explicit reason for change
- angle_packs: universal pack definitions and domain-specific pack metadata

Memory strategy

- Short-term memory: current conversation state and prior follow-up context.
- Long-term memory: saved forecasts, prior outputs, preferred domains, preferred formatting, and historical revision log.
- Optional semantic memory: embeddings over prior analyses to surface similar past questions.

9. MVP build plan

MVP scope

The MVP should answer one broad class of question very well: “What are all the serious angles, scenarios, and indicators behind this question?” It should support politics/geopolitics, markets/crypto, and one lighter domain such as sports or gaming to validate extensibility.

MVP deliverables

- Single-query interface with structured output
- Universal angle library plus three domain packs
- Six-agent prompt workflow
- Scenario scoring and watchlist generation
- Saved history and forecast revision tracking
- Basic retrieval for recent public information and data points

Recommended delivery plan (8 weeks)

Week	Workstream	Deliverables	Owner focus

1	Spec + ontology	Finalize PRD, define universal angle library, define first domain packs	Founder / PM
2	Backend skeleton	Set up API, database schema, run logging, and prompt versioning	Engineer
3	Core prompting	Build normalizer, router, lens analyst, and scenario engine	Engineer + PM
4	Red team + synthesis	Add red-team pass, final synthesis, and response renderer	Engineer
5	Retrieval layer	Add web/news and selected market data retrieval for fresh context	Engineer
6	Memory + revisions	Save runs, scenario scores, and enable follow-up comparison	Engineer
7	QA + eval harness	Create test set, measure coverage and output consistency, fix weak prompts	PM + Analyst
8	Pilot launch	Deploy MVP, onboard a small user cohort, capture failure cases and upgrade priorities	Founder / PM

Initial evaluation harness

- Create a benchmark set of 30 to 50 questions across the priority domains.
- Score each output for lens coverage, internal consistency, usefulness of watchlist, and clarity of change-of-mind conditions.
- Run adversarial prompts that try to force ideology, certainty, or single-factor explanations.

- Compare first-draft outputs against revised outputs after red-teaming; the improvement should be visible.

10. Risks, controls, and next-phase roadmap

Key risks

- Angle explosion: the system may list too many angles and fail to prioritize.
- Narrative masquerading as analysis: elegant wording may hide shallow causal thinking.
- Stale factual context: political, market, and sports questions can degrade quickly without retrieval.
- Prompt sprawl: too many complex prompts can raise cost and make debugging painful.
- False precision: probabilities may look rigorous without deserving the confidence implied.

Controls

- Rank lenses by causal relevance, immediacy, and measurability.
- Force every scenario to include disconfirming signals.
- Use retrieval on freshness-sensitive questions and log sources.
- Keep agent roles narrow and output schemas strict.
- Display confidence separately from probability.

Post-MVP roadmap

- Add automated refresh workflows that update saved forecasts when key indicators move.
- Add richer data feeds for macro, equities, crypto, sports, and gaming telemetry.
- Add user-specific strategic modes such as investor mode, operator mode, and content mode.
- Add portfolio-style dashboards for multiple tracked questions.
- Add team collaboration, notes, and internal debate workflows.

Recommended first build decision

Start with a narrow but serious launch surface: one high-quality interface, three strong domain packs, and a disciplined scenario engine. Do not try to win by breadth first. Win by forcing better thinking than generic chat tools, then expand horizontally.

Appendix A. Starter prompt templates

Normalizer prompt skeleton

```
You are the Normalizer. Rewrite the user’s question into a precise analytical question. Define ambiguous terms. Do not answer the question yet. Return JSON only.

Required fields: normalized_question, definitions, ambiguities, time_horizon, user_objective_assumption.
```

Lens analyst prompt skeleton

```
You are the Lens Analyst. Given the normalized question and selected angle packs, list every
serious analytical lens. Rank them by importance. For each lens, state why it matters, the key
driver, and what evidence would confirm or weaken it. Return JSON only.
```

Scenario engine prompt skeleton

```
You are the Scenario Engine. Build a scenario matrix with base, bull, bear, and wildcard cases.
For each scenario, provide probability, impact, time horizon, leading indicators, disconfirming
signals, and confidence level. Return JSON only.
```

Red-team prompt skeleton

```
You are the Red Team. Your job is to attack the current analysis. Identify hidden assumptions,
missing variables, weak analogies, ideological bias, and ignored tail risks. Suggest how the
probability distribution should change, if at all. Return JSON only.
```

Appendix B. Example query transformation

Example raw query: “Would Donald Trump fail his second election by 2027?”

- Step 1 — normalize: replace the malformed framing with a legally and temporally coherent question.
- Step 2 — define failure: policy failure, popularity collapse, coalition fracture, legislative blockage, legal defeat, or electoral backlash.
- Step 3 — route: politics, law, macroeconomics, social dynamics, narrative/media, and markets.
- Step 4 — generate scenarios and indicators instead of forcing one verdict.

Appendix C. Completeness addendum against source framework

This appendix explicitly captures source-framework elements that were conceptually present in the document but should be stated more directly for execution discipline.

1. Explicit action layer by user archetype

- Investor mode: translate scenarios into exposures, catalysts, positioning implications, and what is likely mispriced.
- Operator mode: translate scenarios into business decisions, sequencing, contingency plans, and resource allocation choices.
- Researcher mode: translate scenarios into open questions, missing evidence, and highest-value next investigations.
- Content mode: translate scenarios into narratives, counter-narratives, and underappreciated framing angles.
- Forecaster / bettor mode: translate scenarios into probability distributions, expected-value opportunities, and timing-sensitive watchpoints.

2. Explicit committee composition

- Political Analyst
- Macro Strategist
- Market Analyst
- Crypto Analyst
- Behavioral Psychologist
- Historian
- Skeptic / Red Team
- Probability Forecaster
- Synthesizer / Chief of Staff

3. Canonical final output structure

- Reframed question
- Definitions
- Relevant lenses
- Key drivers
- Scenario matrix
- Arguments for
- Arguments against
- Hidden variables
- What would change my mind
- Bottom line

4. Worldview principles

- Multi-causal: large outcomes are rarely driven by one variable.

- Incentive-driven: systems run on incentives even when actors talk in values.
- Probabilistic: outputs must present distributions, not false certainty.
- Reflexive: beliefs, narratives, and positioning can change the outcome itself.
- Cross-domain: politics, markets, technology, culture, and society interact continuously.
- Anti-narrative: narratives matter but must never substitute for causal analysis.
- Updateable: every view must remain revisable when evidence changes.

5. Product architecture framing

- Universal core: the same question-normalization, lens-expansion, scenario, and scoring pipeline should apply to any topic.
- Domain plug-ins: specialized angle packs should extend the universal core for politics, crypto, NBA, gaming, and future domains without rewriting the core logic.

6. MVP implementation options

- No-code / low-code MVP
  - Reasoning layer: ChatGPT or Claude
  - Memory layer: Notion or Airtable
  - Freshness layer: Perplexity or web retrieval
  - Automation layer: Zapier or Make
- Serious build path
  - Python backend
  - LLM orchestration layer
  - Vector memory store
  - News and market APIs
  - Scoring database
  - Dashboard frontend